Plan: Vertex Pre-filter for the Per-Vertex Top-K Cycle Enumerator

HyMeKo / HSiKAN

2026-05-11

1 Scope and goal

Add a pluggable *vertex pre-filter* step before the per-vertex DFS in `hymeko_graph::topk_cycles`, so vertices unlikely to contribute informative cycles are excluded from being DFS roots. The lift study on 2026-05-10 measured that **61% of Epinions vertices sit on no balanced 4-cycle reachable from the DFS** — they participate in the enumeration cost (per-vertex BFS pre-pass + DFS startup) but contribute zero output. Skipping them is pure work elimination.

Concrete goal. Cut the Epinions production-config k=5 enumeration wall and peak RSS by $\geq 30\%$ at iso-AUC, with a mechanism that supports multiple filter strategies (degree, triangle, clustering, local sign-entropy, clique size, balanced-reachable) so that future regimes (Slashdot, Bitcoin Alpha/OTC, future datasets) can pick the best filter for their structure.

2 Affected files

All heuristic computation lives in Rust — on Epinions scale (131k vertices, 841k edges) Python triangle/clustering/clique-size loops would be 10-100× slower than the savings. Python only handles env-var dispatch and passes the filter selection to Rust.

- `hymeko_graph/src/vertex_filter.rs` — NEW. Defines the `VertexFilter` trait + 6 concrete impls (`NoFilter`, `DegreeFilter`, `TriangleFilter`, `ClusteringFilter`, `LocalSignEntropyFilter`, `CliqueSizeFilter`, `BalancedReachableFilter`). Each impl returns a `Vec<u32>` of vertex ids to keep.
- `hymeko_graph/src/topk_cycles.rs` — extend `enumerate_top_k_per_vertex_cycles_par(_adaptive)` to accept an optional `&dyn VertexFilter` (or pre-computed `&[u32]` keep-set). Default behaviour: iterate over `0..n_nodes` (no filter). Strictly additive — existing callers see no behaviour change.
- `hymeko_graph/src/lib.rs` — re-export the trait and filter impls.
- `hymeko_py/src/cycles.rs` — thread a `filter_kind: &str` + `filter_params: &PyDict` through the existing PyO3 bindings. Macro-dispatch over the 7 filter kinds at the binding boundary, then pass a typed reference to the Rust enumerator (no `Box<dyn>` indirection in the hot path).
- `signedkan_wip/src/n_tuples.py` — read `HSIKAN_VERTEX_FILTER` env var + threshold env vars; pass them through to the Rust binding as a `(filter_kind, filter_params)` pair. No heuristic computation in Python.
- `signedkan_wip/tests/test_vertex_filter.py` — NEW. End-to-end tests via the PyO3 binding: each filter strategy on a small SBM fixture returns the expected keep-set; AUC parity at `filter_kind="none"`.
- `hymeko_graph/tests/vertex_filter.rs` — NEW. Rust-side unit tests for each `VertexFilter` impl on hand-coded fixtures.

- `hymeko_graph/tests/vertex_prefilter_integration.rs` — NEW. Filtered enumerator output is a subset of unfiltered; bit-identity at `filter_kind="none"`; sequential vs parallel parity under filter.
- `hymeko_graph/benches/vertex_filter_bench.rs` — NEW. Criterion bench: filter computation cost (target < 5 s on Epinions for cheap filters; < 60 s for `balanced_reachable`).

3 CORE.YAML items touched

None. `hymeko_graph`, `hymeko_py`, `signedkan_wip` are not under `CORE.YAML`'s `crates/files/globs`; `tests/**` and `examples/**` are in the `allowlist`.

4 Interface changes

Rust trait + impls

```
// hymeko_graph/src/vertex_filter.rs
pub trait VertexFilter: Send + Sync {
    /// Return the set of vertex ids that should be DFS roots.
    /// `Vec<u32>` ordered ascending.  Empty vec is a valid result.
    fn keep_set(&self, g: &SignedGraph) -> Vec<u32>;
    fn name(&self) -> &'static str;
}

pub struct NoFilter;
pub struct DegreeFilter { pub min_degree: u32 }
pub struct TriangleFilter;
pub struct ClusteringFilter { pub min_cc: f32 }
pub struct LocalSignEntropyFilter { pub min_entropy: f32 }
pub struct CliqueSizeFilter { pub min_clique: u32 }
pub struct BalancedReachableFilter { pub k: usize }

// Composition (AND).
pub struct AndFilter(pub Vec<Box<dyn VertexFilter>>);
```

Per-impl complexity (worst-case Epinions)

Filter	Cost	Wall on Epinions
`NoFilter`	$O(1)$	< 10 ms
`DegreeFilter`	$O(\|E\|)$ via bincount	< 100 ms
`TriangleFilter`	$O(\|E\| \cdot \overline{d})$ via shared adjacency	1–2 s
`ClusteringFilter`	$O(\|E\| \cdot \overline{d})$ same as triangle	1–2 s
`LocalSignEntropyFilter`	$O(\|E\|)$ via per-vertex sign histogram	< 200 ms
`CliqueSizeFilter` (q=3)	$O(\|E\| \cdot \overline{d})$	1–2 s
`CliqueSizeFilter` (q=4)	$O(\|E\| \cdot \overline{d}^2)$	10–30 s
`BalancedReachableFilter`	one-shot k-cycle enum + mark	30–60 s

Per-vertex enumerator integration

```
// hymeko_graph/src/topk_cycles.rs
pub fn enumerate_top_k_per_vertex_cycles_par<P, S, F>(
    g: &SignedGraph, k: usize, m_per_vertex: usize,
    pruner: &P, scorer: &S,
```

```
    filter: &F,                          // NEW: zero-cost generic
) -> Vec<Cycle>
where
    P: Pruner + Send + Sync,
    S: Scorer + Send + Sync,
    F: VertexFilter + ?Sized;

// Legacy entry-point (calls into _par with NoFilter) preserved
// for backwards compatibility with existing callers.
```

PyO3 binding

```
// hymeko_py/src/cycles.rs
#[pyfunction]
fn enumerate_top_k_per_vertex_cycles_signed_rs(
    eu, ev, es, n_nodes, k, m_per_vertex,
    scorer_kind: &str, pruner_kind: &str,
    // NEW (with default empty for backwards compat):
    filter_kind: &str,                  // "none" | "degree" | ...
    filter_params: &PyDict,             // {"min_degree": 2, ...}
) -> (PyArray2<u32>, PyArray1<f32>);

// Macro-dispatched: each (scorer x pruner x filter) combination
// monomorphises a single concrete enumerator call.
```

Python (env-var dispatch only)

```
# signedkan_wip/src/n_tuples.py (no heuristic logic in Python)
filter_kind = os.environ.get("HSIKAN_VERTEX_FILTER", "none")
filter_params = {
    "min_degree": int(os.environ.get(
        "HSIKAN_VERTEX_FILTER_MIN_DEGREE", "2")),
    "min_cc": float(os.environ.get(
        "HSIKAN_VERTEX_FILTER_MIN_CC", "0.05")),
    "min_entropy": float(os.environ.get(
        "HSIKAN_VERTEX_FILTER_MIN_ENTROPY", "0.4")),
    "min_clique": int(os.environ.get(
        "HSIKAN_VERTEX_FILTER_MIN_CLIQUE", "3")),
}
arr, _scores = hymeko.enumerate_top_k_per_vertex_cycles_signed_rs(
    eu, ev, es, n_nodes, k, K, scorer, pruner,
    filter_kind, filter_params,
)
```

Env vars

```
HSIKAN_VERTEX_FILTER = none | degree | triangle | clustering
                     | local_sign_entropy | clique_size
                     | balanced_reachable
                     | "compose:<f1>,<f2>,..."
HSIKAN_VERTEX_FILTER_MIN_DEGREE  = 2     (default)
HSIKAN_VERTEX_FILTER_MIN_CC      = 0.05
HSIKAN_VERTEX_FILTER_MIN_ENTROPY = 0.4
HSIKAN_VERTEX_FILTER_MIN_CLIQUE  = 3
```

5 Test strategy

Unit (Python, `tests/test_vertex_filter.py`)

- Each strategy on a 6-vertex hand-coded fixture; assert exact keep-set.
- Threshold monotonicity: relaxing the threshold returns a superset.
- Composition: AND of two strategies returns intersection.
- Empty-graph edge case: returns `np.empty(0)` cleanly.

Unit (Rust, `tests/vertex_prefilter.rs`)

- `starting_vertices=Some(&[v])` returns cycles starting only at v (canonical-rotation-aware).
- `starting_vertices=None` matches the existing `enumerate_top_k_per_vertex_cycles_par` bit-for-bit.
- Sequential vs parallel parity under filtered starting set.

Integration

- Filtered enumerator output is a *subset* of unfiltered output (same cycles, fewer of them).
- For `strategy="all"` (no filter), output bit-equals existing enumerator.
- AUC parity check on a small SBM fixture: 1-seed AUC delta ≤ 0.005 between filtered (degree+triangle) and unfiltered.

Performance test

- Criterion bench: Epinions k=5, m=128, balance pruner, with vs without vertex pre-filter. Assert filtered wall $\leq 0.7\times$ unfiltered wall (i.e., $\geq 30\%$ reduction).
- Memory: filtered peak RSS $\leq 0.7\times$ unfiltered peak RSS.

6 Performance budget

- Filtered wall: $\leq 70\%$ of unfiltered (Epinions k=5).
- Filtered RSS: $\leq 70\%$ of unfiltered.
- Filter-computation overhead: ≤ 5 seconds for the cheap strategies (degree, triangle); ≤ 60 seconds for balanced_reachable (one-shot enum + mark).
- Output cycle count: filtered $\leq$ unfiltered (subset property).
- AUC delta vs unfiltered (1-seed Epinions abbreviated config): ≤ 0.005. 5-seed paired validation gates promotion to default.

7 Rollback path

- Default `HSIKAN_VERTEX_FILTER=all` preserves existing behaviour bit-for-bit. Setting the env var is opt-in.
- The new Rust function is additive; the existing entry point keeps its signature. Reverting the patch removes only new code.
- All five PyO3 bindings remain backwards-compatible: the new optional argument defaults to `None`.

8 Risk anticipation (CLAUDE.md §2)

Performance-contract preservation

The current per-vertex enumerator passes `m_per_vertex` into the Rust DFS (the per-vertex heap cap). A separate fix landed today (`n_tuples.py:_subsample_arr_to_tuples`) that pushes the *output* cap down to numpy-array level before Python-tuple materialisation. **This plan must preserve both contracts.** The filter parameter does not interact with either; it bounds the set of *starting* vertices, not the per-vertex output count.

Aspirational comments to honour

`n_tuples.py:367-369` says "Push the cap into the enumerator so we never materialise more than `max_cycles` Python tuples in memory." This plan does *not* fix that — the `cap_total` push-down is a separate followup (`project_rust_topk_memory_followup_2026_05_10.md` in auto-memory). Filter pre-pass reduces the *number of vertices* we enumerate from, which indirectly reduces total cycles enumerated, but the per-arity Rust-side memory bound still depends on the `cap_total` fix.

Worst-case input

Production-scale Epinions, k=5, m=128, balance pruner, all 6 arities (`c2,c3,c4,c5,w2,w3`). Filter computed on a graph with 131,828 vertices and 841,372 edges.

What could go wrong

- **Filter is more expensive than the savings**: the cheap strategies (degree, triangle) are bounded $O(|E|)$, $O(|E| \cdot \bar{d})$ — safely under 5s on Epinions. The expensive ones (balanced_reachable) are scoped behind a separate env var with their own perf gate.
- **Filter changes AUC**: keep-set monotonicity test + fixed-strategy="all" bit-parity test guard against accidental behaviour change. AUC parity tested on the smoke and 5-seed gated for promotion.
- **Rayon thread imbalance**: if the keep-set is small (< 100 vertices) the parallel work stealing might starve. Mitigation: log keep-set size at start; if < 1000, suggest a smaller filter.

9 Empty-plan-dir hygiene (CLAUDE.md §2)

This plan dir starts populated. If the work is abandoned at any halt condition (Section 9), the dir gets a `HALTED.md` explaining the abort reason rather than being left empty.

10 Halt conditions

- Filter-computation overhead exceeds budget for the chosen strategy.
- AUC delta on the 1-seed smoke exceeds 0.01 at any strategy (filter is suppressing informative cycles — pivot to less aggressive thresholds).
- Composition logic forces a CORE.YAML edit.

11 Implementation order (v1 / v2)

v1 (this plan)

1. Rust API extension (`starting_vertices: Option<&[u32]>`).

2. PyO3 binding update.
3. `vertex_filter.py` with `degree`, `triangle`, `compose`, `all`.
4. `n_tuples.py` env-var dispatch.
5. All unit + integration + criterion tests.
6. Smoke on Epinions k=5 with `compose:degree>=2+triangle>=1`.

v2 (separate plan, gated on v1 pass)

1. `clustering`, `local_sign_entropy`, `clique_size`, `balanced_reachable` strategies.
2. Per-strategy AUC sweep on production config.
3. 5-seed paired validation for promotion of best strategy to default.
4. **Tiered concentric caps (FPN P-levels)**: m_v as a step-function of vertex centrality, not linear in degree. P5 (top 0.1% hubs): m_v=1024; P4 (1%): m_v=256; P3 (5%): m_v=64; P2 (20%): m_v=16; P1 (peripheral): skipped. Total cycles drop $\sim$3$\times$ vs fixed m=128; signal density per row rises sharply.

v3 (Mixture 2: per-tier algorithm selection)

Algorithmic complexity matches structural importance. Each FPN tier gets its own DFS strategy — the strategy whose heap-fill rate matches the tier's m_v best. Composes naturally with v2's tiered caps and the user's cognitive-propagation framing (refined planning at the top, bulk reactive enumeration at the bottom).

Tier	m_v	Algorithm	Justification
P5 (top 0.1%)	1024	A*-best-first + shared threshold	high m_v rarely fills $\rightarrow$ A* finds top cycles fastest
P4 (top 1%)	256	A*-warmup $\rightarrow$ DFS-ABB hybrid	medium heap, both phases help
P3 (top 5%)	64	DFS-ABB	heap fills fast, ABB threshold tightens
P2 (top 20%)	16	DFS no-pruning	heap so small that ABB overhead > gain
P1 (periph.)	0	skip (vertex pre-filter)	no cycles contributed

The Rust trait surface for v3:

```
pub trait PerVertexStrategy: Send + Sync {
    fn enumerate_from(&self, g: &SignedGraph,
                      v: u32, k: usize, m_v: u32,
                      pruner: &dyn Pruner,
                      scorer: &dyn Scorer,
                      shared_threshold: &AtomicF64,
                      ) -> Vec<HeapEntry>;
}

pub struct AStarStrategy { /* priority queue, shared threshold */ }
pub struct AStarToDfsHybridStrategy { warmup_size: usize }
pub struct DfsAbbStrategy;
pub struct DfsNoPruneStrategy;

pub struct TieredDispatcher {
    pub tiers: Vec<(u32 /*m_v*/, Box<dyn PerVertexStrategy>)>,
    pub tier_of_vertex: Vec<usize>,  // n_nodes-long lookup
}
```

Note: the v3 trait is per-vertex, not per-graph — the dispatch fires once per starting vertex. Rayon par_iter over starting vertices, each grabbing its own tier's strategy. Shared threshold is an `AtomicF64` (f64-as-u64-bits) so all rayon workers prune against the same top-K-min, regardless of which strategy produced the threshold.

v3+ (Mixture 3: Pareto-frontier multi-scorer union)

Orthogonal to v3's per-tier algorithm. Within each strategy, run multiple scorers in parallel and union their top-K's — the output is Pareto-diverse rather than single-axis-optimal.

Tier	Strategy (v3)	Scorer set (v3+)
P5	A* + shared thresh	`fraction_negative` only
P4	A* → DFS-ABB	`fraction_negative` + `local_sign_entropy`
P3	DFS-ABB	`fraction_negative` + `low_root` + `sign_product_abs`
P2	DFS no-prune	no scorer (all retained)

Mechanic per vertex v in tier T:

1. Let scorer set be $S_T = \{s_1, \ldots, s_{|S_T|}\}$.
2. For each s_i: enumerate top-$\lceil m_v/|S_T| \rceil$ cycles ranked by s_i (using T's algorithm strategy).
3. Union the per-scorer outputs, deduplicating by canonical cycle key (sorted vertex tuple + edge-sign tuple).
4. Output: at most m_v cycles, Pareto-diverse across scorers.

The Rust trait extension for v3+:

```
pub struct TierConfig {
    pub m_v: u32,
    pub strategy: Box<dyn PerVertexStrategy>,
    pub scorers: Vec<Box<dyn Scorer>>,   // NEW
}

impl TierConfig {
    pub fn enumerate_from(&self, g: &SignedGraph, v: u32,
                          k: usize, ...) -> Vec<Cycle> {
        let per_scorer = (self.m_v as usize)
                         .div_ceil(self.scorers.len());
        let mut all = HashMap::new();
        for scorer in &self.scorers {
            for cycle in self.strategy.enumerate_from(
                    g, v, k, per_scorer as u32, scorer, ...) {
                all.entry(cycle.canonical_key())
                   .or_insert(cycle);
            }
        }
        all.into_values().take(self.m_v as usize).collect()
    }
}
```

Why this is good (justification):

- Each scorer captures a different aspect of cycle quality (balance ratio, root stability, sign-product abs). No single scorer dominates the M_e matrix.
- Pareto-diverse cycle pool reduces overfitting to any one balance metric — protects against the scorer-bias failure mode observed in the 2026-05-10 ABB-vs-per-vertex smoke (signal-only global K=10 000 returned 10 000 score-1.0 cycles; zero variance in M_e).
- Maps to multi-objective optimisation literature; the union step is an approximation of the non-dominated front.
- Composes cleanly with v3: each tier's strategy still fires; v3+ wraps it in a multi-scorer outer loop.

Caveat: per-scorer enumeration cost is multiplicative ($|S_T|\times$ baseline). Wall budget per tier:

- P5: $|S_5| = 1$ (no extra cost; A* already memory-bounded)

- P4: $|S_4| = 2$ ($\sim 2\times$ DFS work, $\sim 2\times$ wall)
- P3: $|S_3| = 3$ ($\sim 3\times$ DFS work, but DFS-ABB at m=64 is cheap; expected $\leq 30\%$ tier-3 wall increase)

Total expected wall vs single-scorer v3: +10–20% (mostly P3).

v4 (MSG/SSG axiom layer, separate plan)

MSG preprocessor restricts (V, E) to the balance-feasible substructure. SSG enumerates minimally balanced cycles within MSG. Both can drop in as a substrate for v3's per-tier strategies — i.e., v3 strategies enumerate *within MSG*, not against $0..n\_nodes$.

Composed picture (v1 + v2 + v3 + v3+ + v4)

1. **MSG** (v4): restrict (V, E) to balance-feasible substructure.
2. **Vertex prefilter** (v1): inside MSG, drop low-degree / no-triangle / etc. vertices.
3. **FPN tier assignment** (v2): bin remaining vertices P5..P2 by centrality.
4. **Per-tier algorithm** (v3): each tier uses the DFS variant matched to its m_v.
5. **Per-tier multi-scorer Pareto union** (v3+): within each strategy, union top-K across multiple scorers.
6. **SSG minimality filter** (v4, optional): downstream, retain only balance-minimal cycles from the v3+ output.

This is the cognitive-propagation architecture in cycle-generator form: refined planning (A* + multi-scorer) at the top tiers, bulk reactive enumeration (DFS-no-prune) at the bottom, with structural axiom layers (MSG/SSG) bracketing the whole.

12 Rollback path (revisited)

1. Single `git revert` of the implementation commit removes the Rust API addition, the PyO3 binding update, and the Python module.
2. Existing callers (yesterday's Slashdot/BitcoinOTC scripts) are untouched: they don't set `HSIKAN_VERTEX_FILTER`, so the default `all` branch fires — bit-equivalent to the current enumerator.